Teoria e Aplicação de Grafos

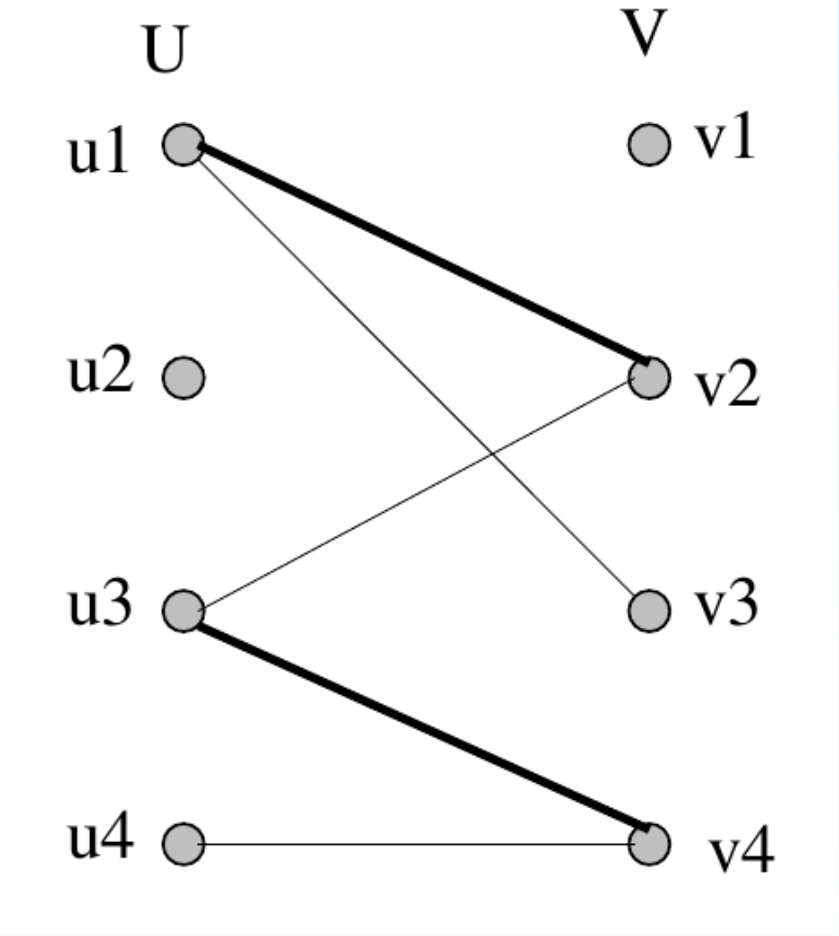
Roteiro da aula:

- **Emparelhamentos Máximos em Grafos (Geral)**
- **Caminhos M-alternado e M-aumentado**
- **Teorema de Berge**
- **Teorema de Tutte**
- **O problema geral de alocação/atribuição**
- **Algoritmo Húngaro**
- **Exemplo**

Definições

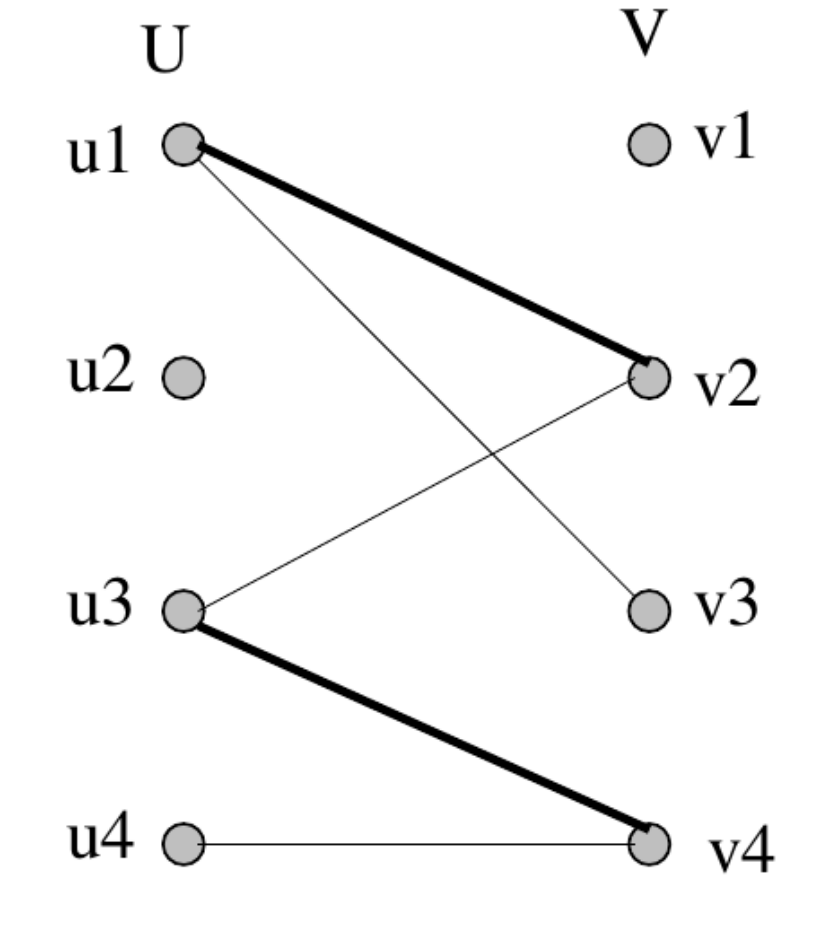
- **Caminho M-alternado: Um caminho em G cujas arestas se alternam em *M* e não em *M*.**
- **Um vértice é M-emparelhado se for incidente a uma das arestas de *M*. Caso contrário é M-não-emparelhado.**
- **Caminho M-aumentado é um caminho alternado que une dois vértices terminais M-não-emparelhados.**
  - **Nem todas as arestas de M**
  - **Sempre começa e termina com uma aresta não em *M***

Exemplos



Emparelhamento M

Exemplos

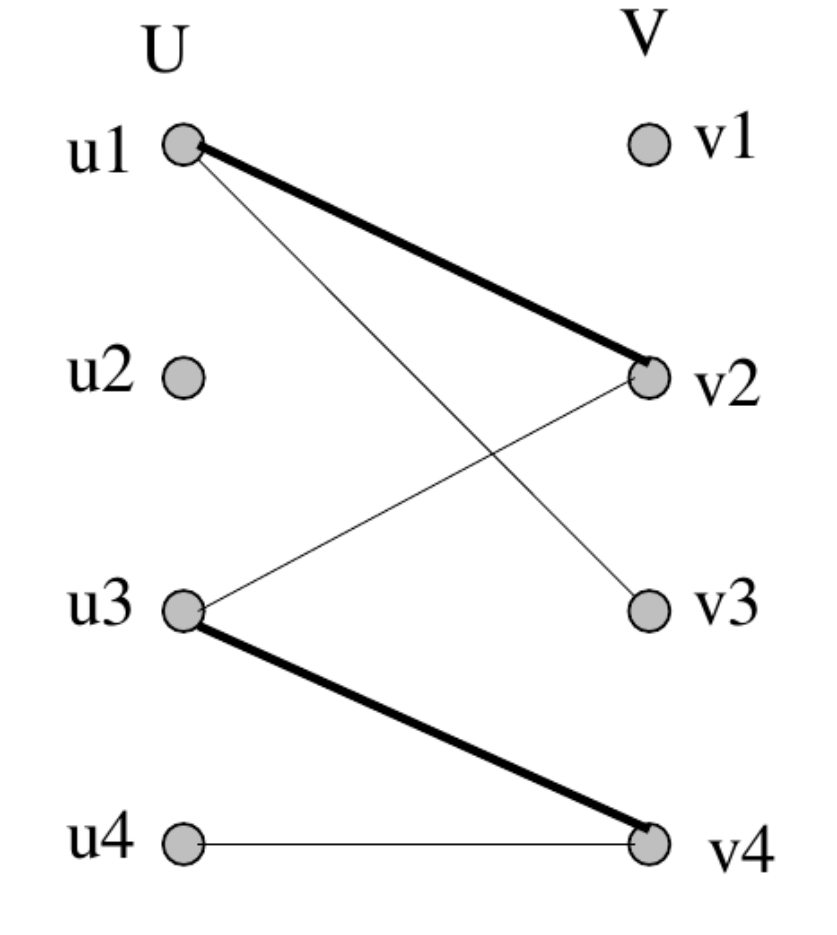


u_2, u_4 livres

u_1, u_3 emparelhados

Emparelhamento M

Exemplos



Emparelhamento M

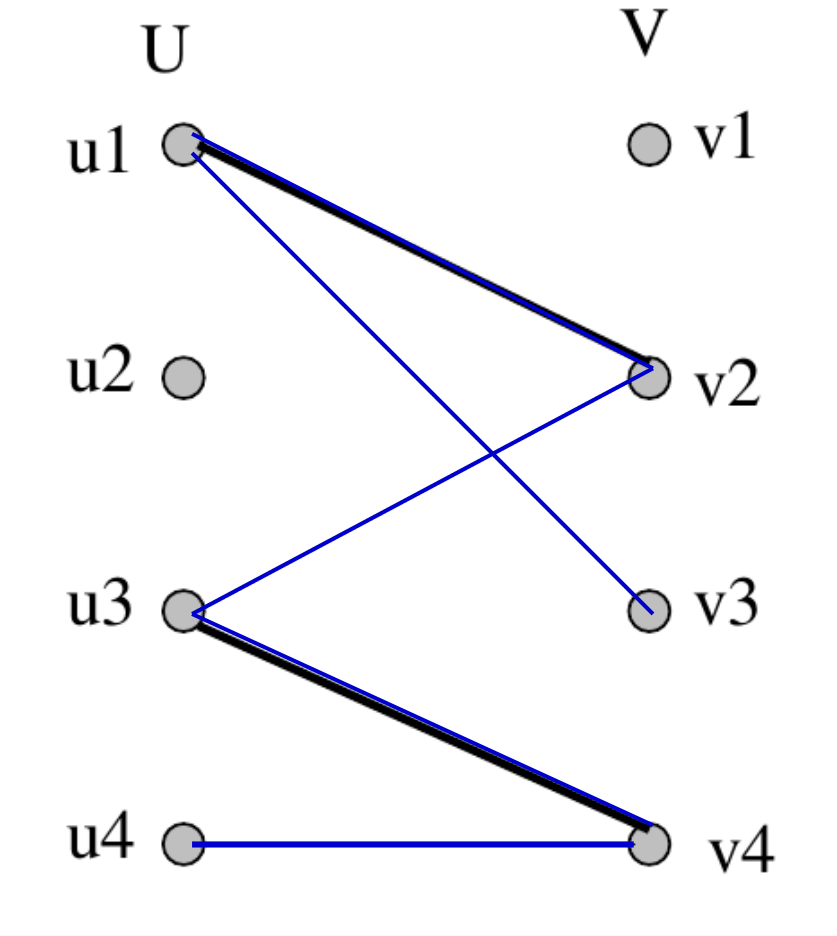
u_2, u_4 livres

u_1, u_3 emparelhados

$u_4, v_4, u_3, v_2, u_1, v_3$

Caminho M-alternado

Exemplos



Emparelhamento M

u_2, u_4 livres

u_1, u_3 emparelhados

$u_4, v_4, u_3, v_2, u_1, v_3$

Caminho M-alternado

Como os dois vértices finais eram livres, este também é M-aumentado

Teorema de Berge

- **Teorema de Emparelhamento de Berge:**

  Um emparelhamento *M* em um grafo G é máximo *se, e somente se,* não houver nenhum caminho M-aumentado em G.

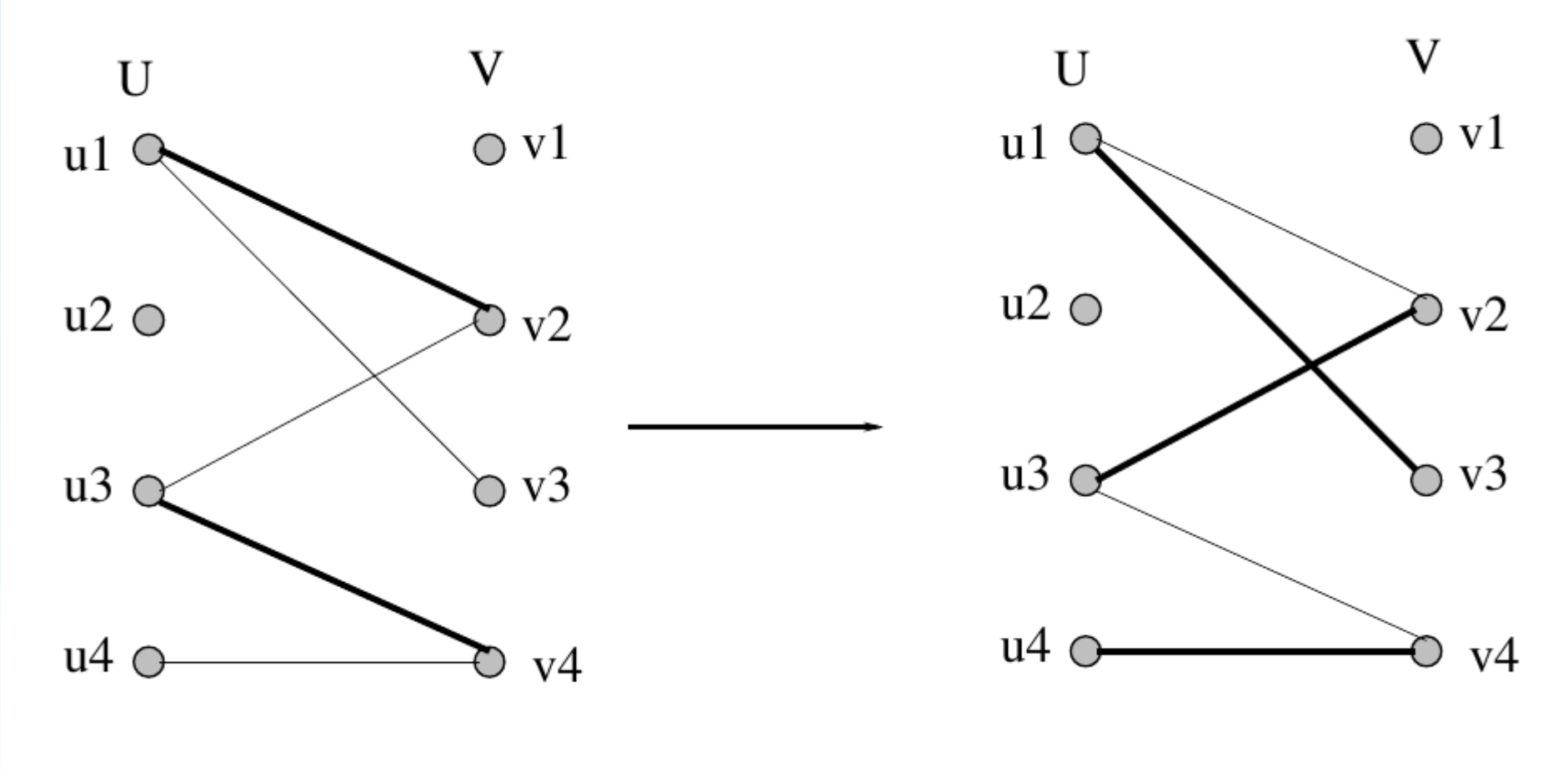
Teorema de Berge

- **Teorema de Emparelhamento de Berge:**

  Um emparelhamento *M* em um grafo G é máximo *se, e somente se,* não houver nenhum caminho M-aumentado em G.

- **Ou, se um emparelhamento M tem um caminho M-aumentado podemos conseguir um emparelhamento maior (máximo) M' trocando as arestas no caminho aumentado.**

Exemplo: Teorema de Berge



M

M’

Teorema de Tutte

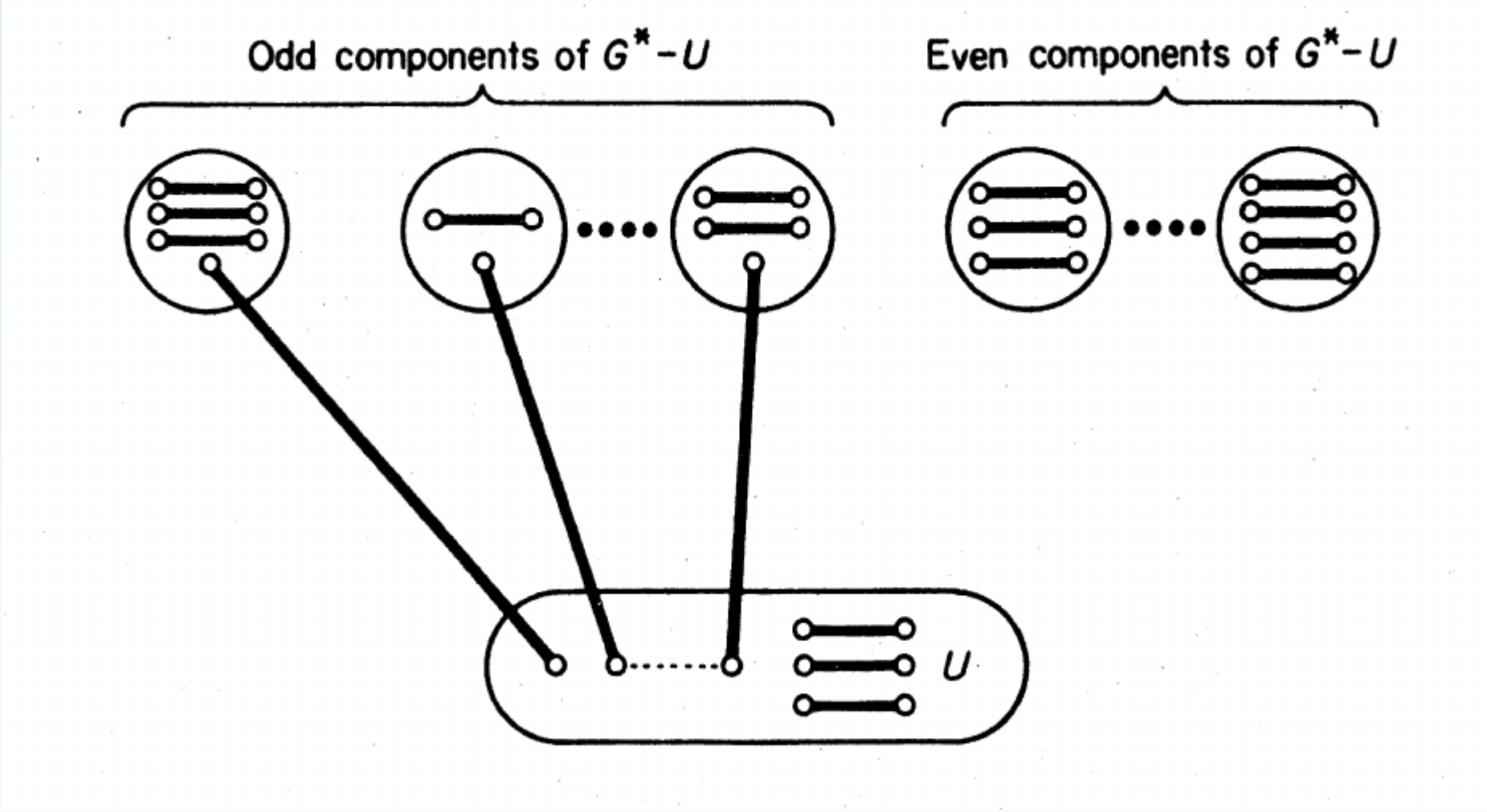
- **Teorema de Emparelhamento de Tutte:**

  Um grafo genérico G possui um emparelhamento perfeito se, e somente se, para todos

$S \subseteq V$, $oc(G - s) \leq |S|$

oc é número de componentes com cardinalidade ímpar

Exemplo: Teorema de Tutte



Bondy & Murty, 1982.

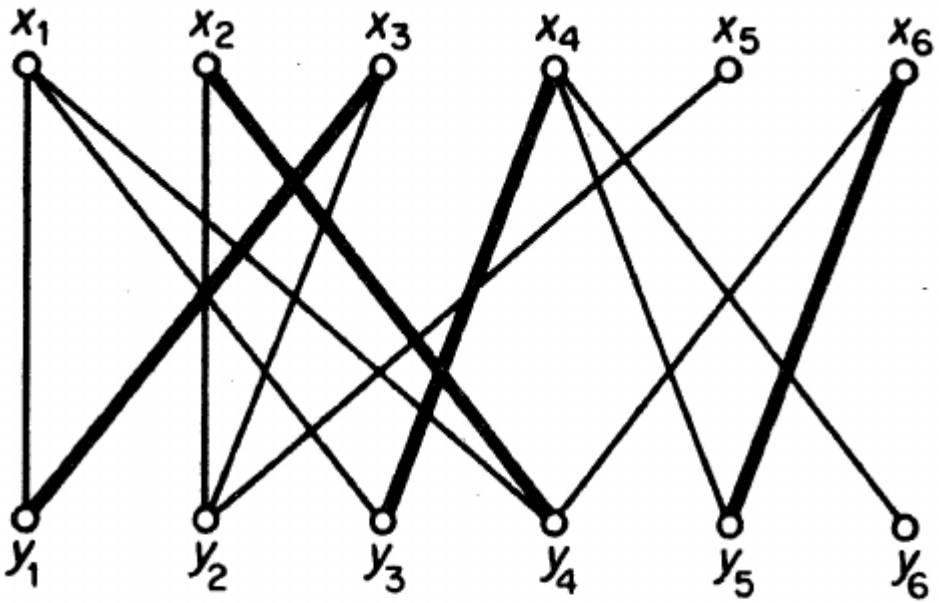
O problema da alocação de pessoal

- **Dado qualquer grafo bipartido G, com bipartição (X,Y),** $X = \{x_1, x_2, \ldots, x_n\}, \quad Y = \{y_1, y_2, \ldots, y_n\}$ **ou encontra um emparelhamento que satura todo vértice em X, ou encontra um subconjunto S de X, tal que** $|N(S)| < |S|$ **(i.e. emparelhamento não perfeito)**

Algoritmo: ideia básica

- Iniciar com um emparelhamento qualquer M
- Se M satura todo vértice em X, fim.
- Se não, escolher um vértice não saturado u, em X, e buscar por um caminho M-aumentado com origem em u.

Algoritmo: ideia básica

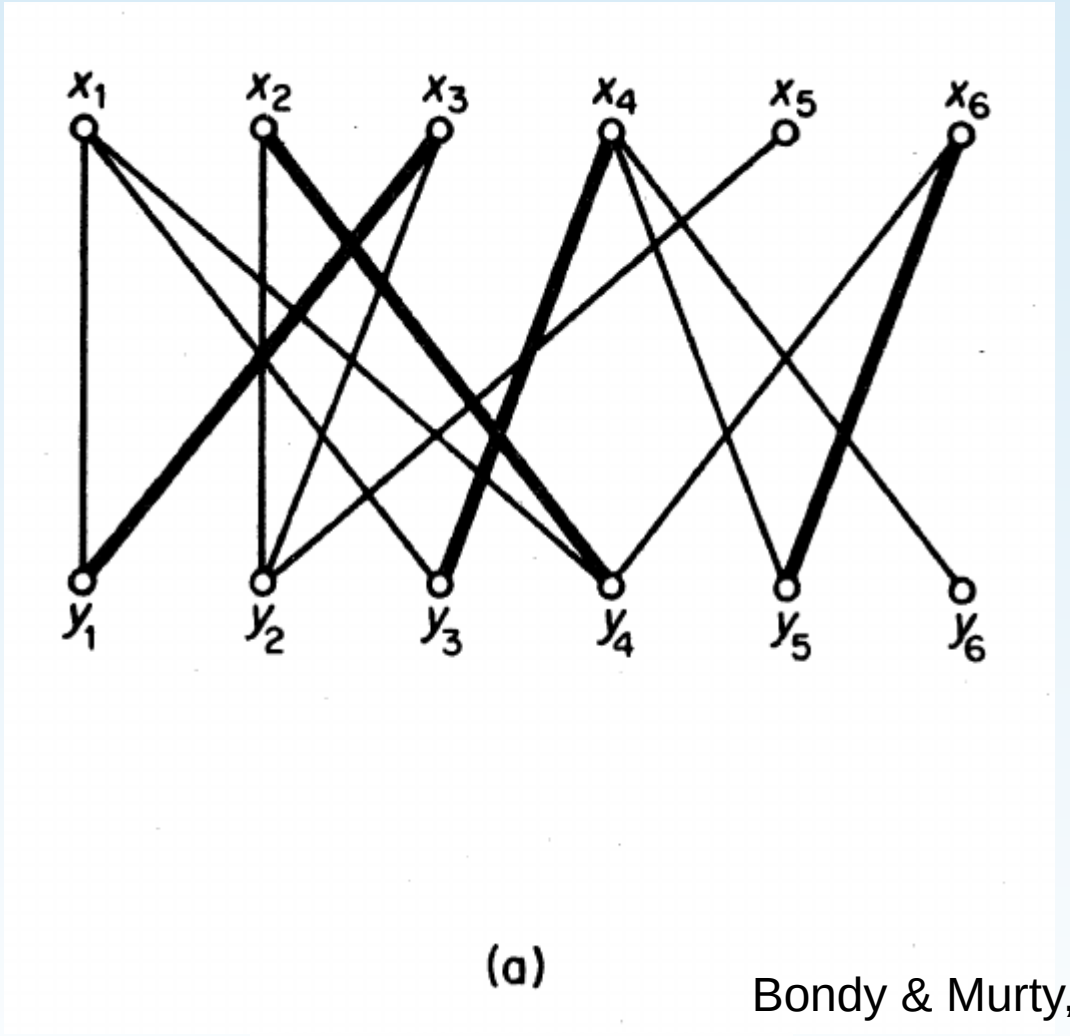


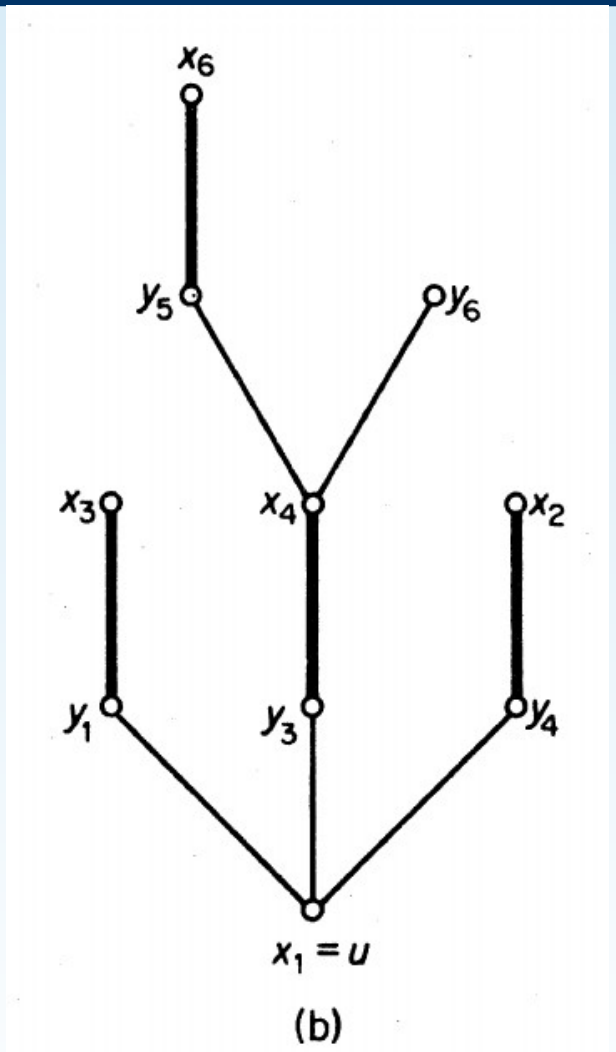
(a)

(a) A matching M in G;

Bondy & Murty, 1982.

Algoritmo: ideia básica





Bondy & Murty, 1982.

(*a*) A matching M in G;

(*b*) an M-alternating tree in G

Algoritmo: ideia básica

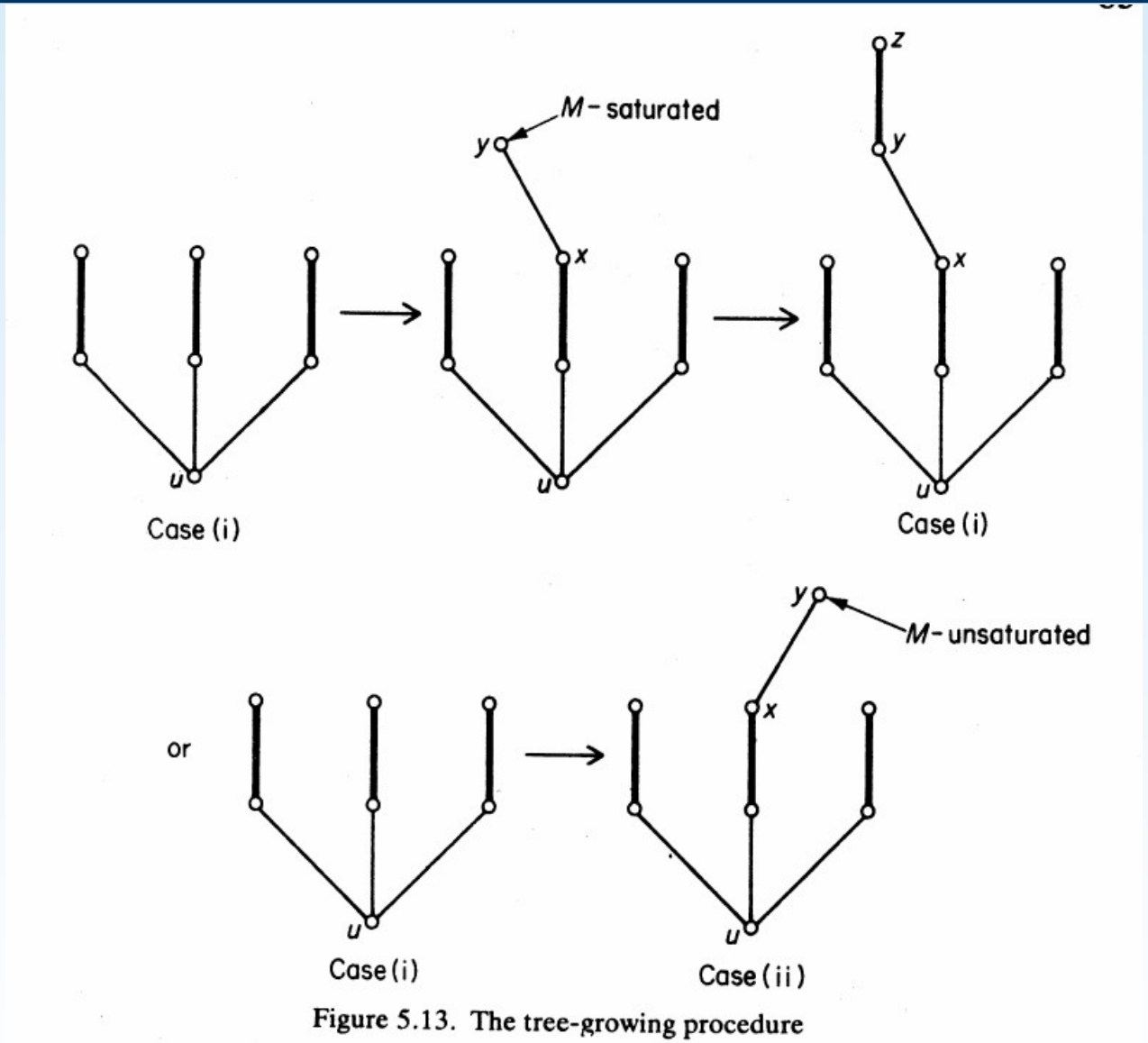


Figure 5.13. The tree-growing procedure

Bondy & Murty, 1982.

Emparelhamento com valores (Exemplo)

Suponha que uma empresa possua 3 vendedores em 3 cidades (S.Paulo, Brasília e B.Horizonte), e que a empresa precisa enviar vendedores a 3 outras cidades (Rio, Cuiabá e Florianópolis). Os custos das passagens estão marcados na tabela. Quem mandar a qual lugar?

De/Para	Rio	Cuiabá	Florianópolis
S.Paulo	250	400	350
Brasília	400	600	350
B.Horizonte	200	400	250

Exemplo

Matriz de custo

$$\begin{bmatrix} 250 & 400 & 350 \\ 400 & 600 & 350 \\ 200 & 400 & 250 \end{bmatrix}$$

Exemplo

Caso seja essa a escolha

$$\begin{bmatrix} \boxed{250} & 400 & 350 \\ 400 & \boxed{600} & 350 \\ 200 & 400 & \boxed{250} \end{bmatrix}$$

Exemplo

Caso seja essa a escolha

$$\begin{bmatrix} \boxed{250} & 400 & 350 \\ 400 & \boxed{600} & 350 \\ 200 & 400 & \boxed{250} \end{bmatrix}$$

Custo total seria

$$\$250 + \$600 + \$250 = \$1100.$$

Exemplo

Caso seja essa a escolha

$$\begin{bmatrix} \boxed{250} & 400 & 350 \\ 400 & 600 & \boxed{350} \\ 200 & \boxed{400} & 250 \end{bmatrix}$$

Custo total seria

$$\$250 + \$350 + \$400 = \$1000.$$

Exemplo

Após checar todos os seis conjuntos diferentes possíveis

$$\begin{bmatrix} 250 & \boxed{400} & 350 \\ 400 & 600 & \boxed{350} \\ \boxed{200} & 400 & 250 \end{bmatrix}$$

Menor custo total seria

$$\$400 + \$350 + \$200 = \$950.$$

Exemplo

Tentativa e erro (i.e. força bruta) funcionaria para poucas cidades.
Mas, e se o número n de cidades fosse maior?

Exemplo

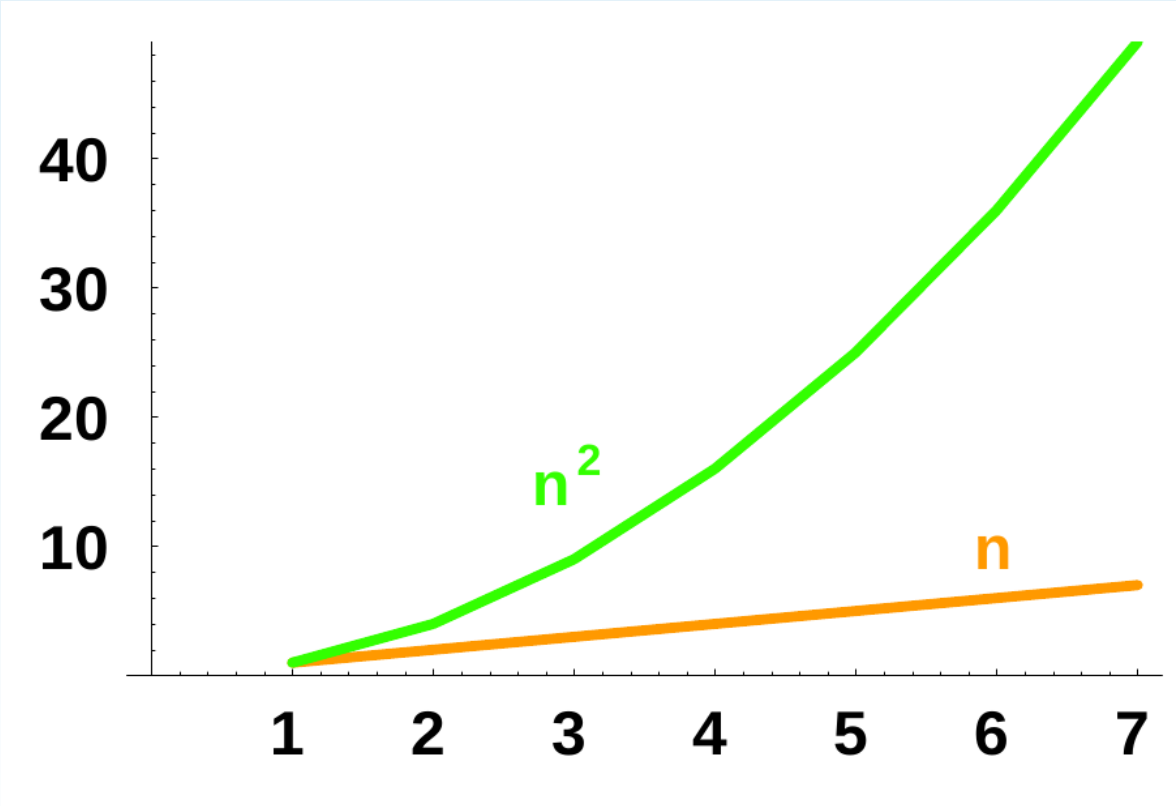
Tentativa e erro (i.e. força bruta) funcionaria para poucas cidades.
Mas, e se o número **n** de cidades fosse maior?

Há **n!** maneiras de alocar **n** recursos a **n** tarefas.

Exemplo

Tentativa e erro (i.e. força bruta) funcionaria para poucas cidades.
Mas, e se o número **n** de cidades fosse maior?

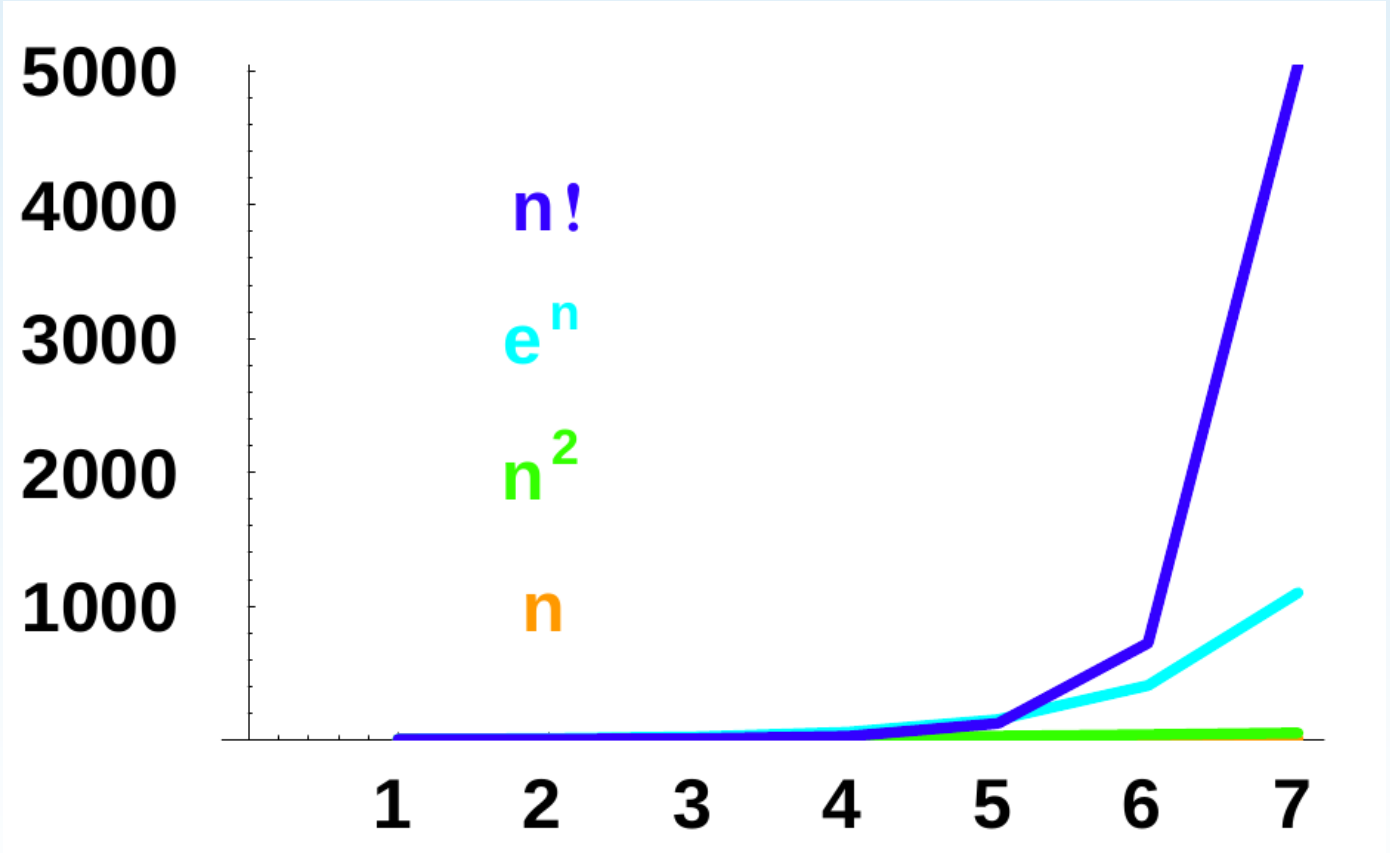
Há **n!** maneiras de alocar **n** recursos a **n** tarefas.



Exemplo

Tentativa e erro (i.e. força bruta) funcionaria para poucas cidades.
Mas, e se o número **n** de cidades fosse maior?

Há **n!** maneiras de alocar **n** recursos a **n** tarefas.



Problema Geral de alocação/atribuição

- n indivíduos (i=1, 2, …, n)
- n tarefas (j=1, 2, …, n)

- custo c_{ij}, custo do indivíduo i fazer a tarefa j
- Como atribuir tarefas aos indivíduos para minimizar o custo total ?

- Valoração r_{ij} indica a qualidade do trabalho
- Como atribuir tarefas aos indivíduos de forma a maximizar a valoração total?

O Algoritmo Húngaro

- **Dois pesquisadores húngaros: Dénes König (1936) e Jenő Egerváry (1931)**

- **Referências:**
- **Harold W. Kuhn, "The Hungarian Method for the assignment problem", *Naval Research Logistic Quarterly*, 2:83-97, 1955.**

- **J. Munkres, "Algorithms for the Assignment and Transportation Problems", *Journal of the Society of Industrial and Applied Mathematics*, 5(1):32-38, 1957.**

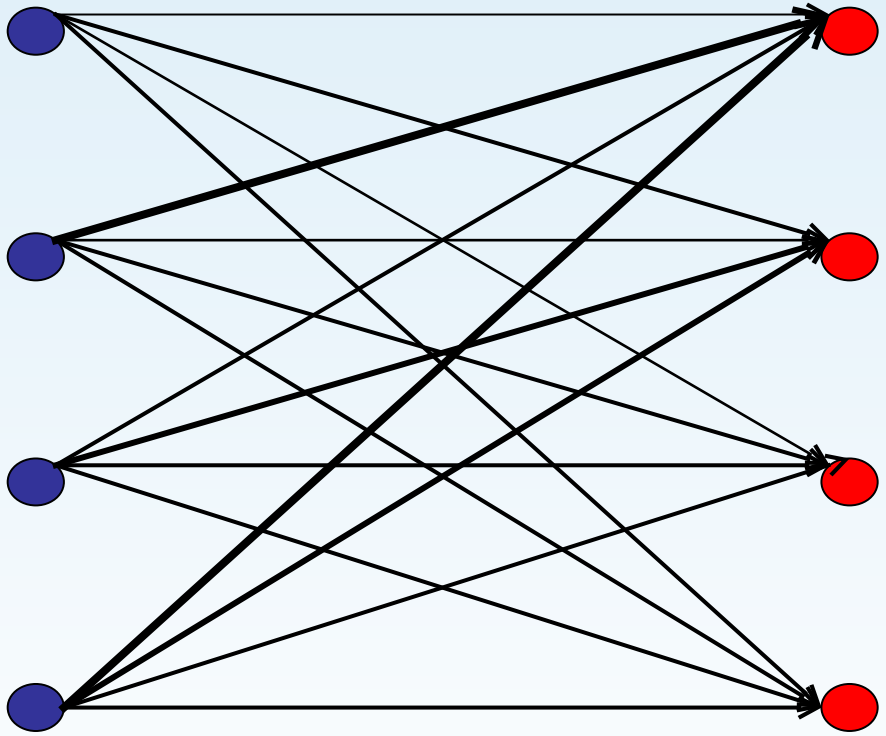
O Algoritmo Húngaro

- **Forma geral, matriz de custo C, ou grafo bipartido.**

$$C = \begin{bmatrix} c_{1,1} & c_{1,2} & \cdots & c_{1,n} \\ c_{2,1} & c_{2,2} & \cdots & c_{2,n} \\ \vdots & \vdots & & \vdots \\ c_{n,1} & c_{n,2} & \cdots & c_{n,n} \end{bmatrix}.$$

O Algoritmo Húngaro

- **Forma geral, matriz de custo C, ou grafo bipartido.**



O Algoritmo Húngaro

- **Exemplo: Determinar como trabalhos sejam passados ao sistema de forma a minimizar tempos de execução:**

		Job 1	**Job 2**	**Job 3**	**Job 4**
Machine 1		14	5	8	7
Machine 2		2	12	6	5
Machine 3		7	8	3	9
Machine 4		2	4	6	10

O Algoritmo Húngaro

- **Duas observações:**

O Algoritmo Húngaro

- **Duas observações:**
  - **Adicionar uma constante a qualquer linha ou coluna não muda a solução → por C**

O Algoritmo Húngaro

- **Duas observações:**
  - **Adicionar uma constante a qualquer linha ou coluna não muda a solução → por C**
  - **Se C for não negativa e $\sum c_{ij}x_{ij} = 0$ então X é uma solução.**

O Algoritmo Húngaro

- **Duas observações:**
  - **Adicionar uma constante a qualquer linha ou coluna não muda a solução → por C**
  - **Se C for não negativa e $\sum c_{ij}x_{ij} = 0$ então X é uma solução.**

- **2 zeros em C serão independentes se aparecerem em linhas e colunas diferentes.**

O Algoritmo Húngaro

Um conjunto de elementos de uma matriz é dito independente se não houver dois deles na mesma linha ou coluna.

- **Teorema de König:**

  Se C é uma matriz e m é o número de elementos zero de C, então há m linhas que contém todos os elementos zeros de C.

O Algoritmo Húngaro

1. **De cada (linha ou coluna) subtrair seu elemento mínimo;**

O Algoritmo Húngaro

1. De cada (linha ou coluna) subtrair seu elemento mínimo;
2. Encontrar um conjunto máximo de N' zeros mutuamente independentes;

O Algoritmo Húngaro

1. De cada (linha ou coluna) subtrair seu elemento mínimo;
2. Encontrar um conjunto máximo de N' zeros mutuamente independentes;
3. *se* N' = N esses zeros foram encontrados: retornar seus índices e parar;

O Algoritmo Húngaro

1. De cada (linha ou coluna) subtrair seu elemento mínimo;
2. Encontrar um conjunto máximo de N' zeros mutuamente independentes;
3. *se* N' = N esses zeros foram encontrados: retornar seus índices e parar;

   caso contrário: cobrir todos zeros em W com N' (linhas ou colunas) e encontrar o valor mínimo não coberto; subtraí-lo dos elementos não cobertos, e adicionar a todos os elementos duplamente cobertos; voltar para passo 2.

O Algoritmo Húngaro

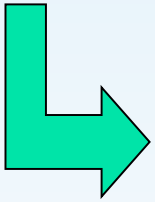
- **Exemplo: Determinar como trabalhos sejam passados ao sistema de forma a minimizar tempos de execução:**

		Job 1	**Job 2**	**Job 3**	**Job 4**
Machine 1		14	5	8	7
Machine 2		2	12	6	5
Machine 3		7	8	3	9
Machine 4		2	4	6	10

O Algoritmo Húngaro

		Job 1	Job 2	Job 3	Job 4
Machine 1		14	**5**	8	7
Machine 2		**2**	12	6	5
Machine 3		7	8	**3**	9
Machine 4		**2**	4	6	10

Passo 1



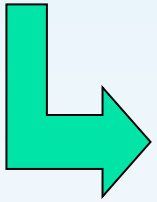
Redução por linha

		Job 1	Job 2	Job 3	Job 4
Machine 1		9	0	3	2
Machine 2		0	10	4	3
Machine 3		4	5	0	6
Machine 4		0	2	4	8

O Algoritmo Húngaro

		Job 1	Job 2	Job 3	Job 4
Machine 1		9	**0**	3	**2**
Machine 2		**0**	10	4	3
Machine 3		4	5	**0**	6
Machine 4		**0**	2	4	8

Passo 1



Redução por coluna

		Job 1	Job 2	Job 3	Job 4
Machine 1		9	0	3	0
Machine 2		0	10	4	1
Machine 3		4	5	0	4
Machine 4		0	2	4	6

O Algoritmo Húngaro

Passo 2

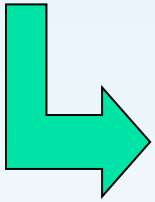
		Job 1	Job 2	Job 3	Job 4
Machine 1		9	0	3	0
Machine 2		0	10	4	1
Machine 3		4	5	0	4
Machine 4		0	2	4	6

encontramos 3<4 linhas/colunas, continuar para passo 3

O Algoritmo Húngaro

		Job 1	Job 2	Job 3	Job 4
Machine 1		9	0	3	0
Machine 2		0	10	4	**1**
Machine 3		4	5	0	4
Machine 4		0	2	4	6

Passo 3



		Job 1	Job 2	Job 3	Job 4
Machine 1		**10**	0	3	0
Machine 2		0	**9**	**3**	**0**
Machine 3		**5**	5	0	4
Machine 4		0	**1**	**3**	**5**

O Algoritmo Húngaro

Atribuições Zero

Passo 2 novamente

		Job 1	Job 2	Job 3	Job 4
Machine 1		**10**	0	3	0
Machine 2		0	**9**	**3**	**0**
Machine 3		**5**	5	0	4
Machine 4		0	**1**	**3**	**5**

Encontramos 4 linhas/colunas,
então atribuição ótima e parar

O Algoritmo Húngaro

		Job 1	Job 2	Job 3	Job 4
Machine 1		10	**0**	3	**0**
Machine 2		**0**	9	3	**0**
Machine 3		5	5	**0**	4
Machine 4		**0**	1	3	5

final

Atribuição ótima

$$x_{12}=1, x_{33}=1, x_{41}=1, x_{24}=1$$

Conjunto máximo de zeros independentes

O Algoritmo Húngaro

$O(V^3)$

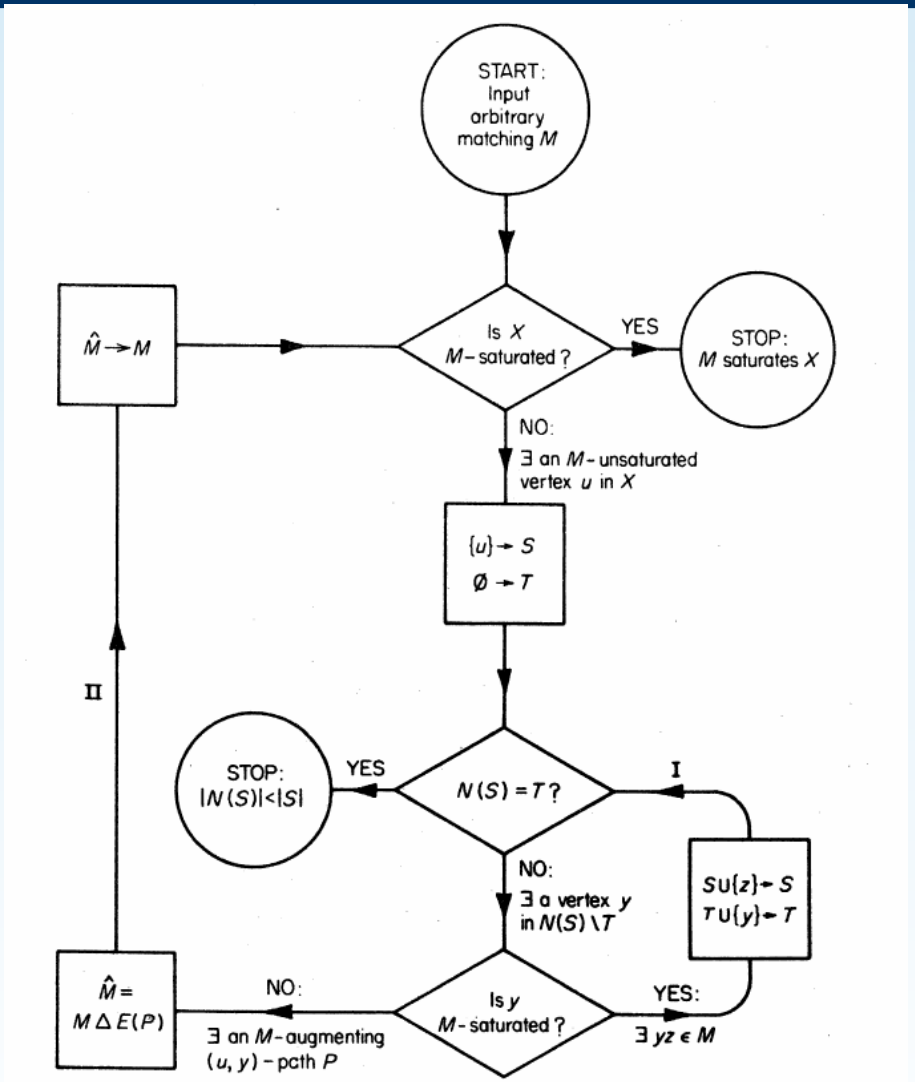


Figure 5.15. The Hungarian method

Bondy & Murty, 1982.

Referências Bibliográficas

- Bogart, K. & Stein, C. & Drysdale, R. Discrete Mathematics for Computer Science. Key College Pub., 2006.
- Bondy, J. & Murty, *Graph Theory with Applications*, Elsevier, 1982.
- Lehman, E. Leighton, F.T. & Meyer, A.R. *Mathematics for Computer Science*, 2016.
- Sedgewick, R. & Wayne, K. *Algorithms* (4th ed.), Addison-Wesley, 2011.
- Wilson, R. & Watkins, J. *Graphs: An Introductory Approach.* John Wiley, 1990.